Touch & feel the modern Java

Gireesh Punathil, IBM
https://www.linkedin.com/in/gireeshpunathil

Agenda

- Java v1.0: premises and paradigms
- Shift in paradigms and workloads
- Java v25.0: response to the shift
- Key takeaways

#10: Performance

v1

Patient Java

- Interpret, warm up, profile, optimize
- Java notoriously slow to start
- Interpretation, class loading, JIT all these add to the startup delay

shift

Tight competition in Cloud functions and microservices

- Cloud services need to be quick and small
- Warmup unacceptable for ephemeral workloads
- Latency and footprint both important as throughput

v25

Nimble Java

- GraalVM (Hotspot)
- Instant ON (Semeru)
- AOT compilation
- Small footprint JRE

JEP 295

#9: Security

v1

Sandboxed Java

- Security manager with classic algorithms
- Every bytecode subjected to security clearance
- Asymmetric key encryption was sufficient

shift

Applet use cases diminished, Major shift in threat model

- Applets banned by browsers
- Code is trusted by default
- Protection is established through external means (e.g., container level)

v25

Post Quantum Java

- Applets deprecated
- Security Manager deprecated
- Quantum safe encryption algorithms - PQC

JEP 411

#8: Diagnostics

v1

Opaque Java

- Stack trace was enough. May be some logging too
- Thread dump, heap dump, core dump
- Verbose gc, verbose class
- System.out.println

shift

Observability central to modern workloads

- Need Performance profiling
- Need open tooling
- Must diagnose from in and out, in production, and under load

v25

Observable Java

- JVMTI (tooling interface)
- JMX (live metrics control)
- Async stack trace (live profiling)
- JFR (360^0 monitoring)

JEP 328 **JEP 349**

#7: Native interoperability

v1

Bridged Java

- JNI for rare cases
- Native calls infrequent
- Slow and verbose
- Rigid and fragile

shift

AI/ML systems operate at native strength

- Tensor ops are native
- Needs faster switch
- Needs less verbose calls
- Needs off-heap access

v25

Integrated Java

- FFI replacing JNI
- Eliminates JNI overhead
- Fast, direct access to C
- Massive performance gain in numerical computing

JEP 454

#6: Parallelism

v1

Scalar Java

- No SIMD use case
- No SIMD APIs
- JIT leverages it transparently if needed
- Loops enough in most cases

shift

ML, crypto, DB: all rich with SIMD use case

- Workload demand for heavy parallelism
- Linear loops highly inefficient
- Most H/Ws support SIMD

v25

Vector Java

- Explicit vector APIs
- Vector primitives and operations for high performant transactions
- Direct Vector computations

JEP 338 **JEP 460**

#5: Concurrency

v1

Heavyweight Java

- Java thread == OS thread
- Expensive in terms of memory and latency
- Manage with thread pools and life cycle control

shift

Cloud and event driven services need simple and faster threading primitives

- Need to scale massively
- Should use less resources
- Should spin up fast

v25

Lightweight Java

- Cheap user level threads
- Concurrency under structured scopes
- Comprehensive API set in Executor services

JEP 444 **JEP 505**

#4: Memory model

v1

Stop-the-world Java

- Apps set explicit heap sizes: -Xms / -Xmx
- Change in workload means change in heap tuning
- Everything is cooked in the Java heap

shift

Workload types and deployment targets need adaptive heap

- Containerized workloads
- Mutli-tenant hosts
- Off-heap data

v25

Pause-less Java

- Auto-heap sizing
- Give back unused memory
- G1 & ZGC (Hotspot)
- Balanced (Semeru)

JEP 346

#3: Object Orientation

v1

Object-oriented Java

- World modeled as objects
- Inheritance
- Abstraction
- Data bound to behavior

shift

World modeled as data with pure transformations

- Raw immutable data
- Data unbound to behavior
- Pattern based logic

v25

Data-oriented Java

- Record patterns for raw data
- Pattern matching as main logic
- Value types for data-first

JEP 401

#2: Platform independence

v1

Portable Java

- Abstract OS and H/W
- WORA
- Portability
- Productivity

shift

WORA is a standard now; but also know the execution env

- Container awareness
- Resource efficiency
- Dynamic scaling

v25

Cloud-native Java

- modularity
- cgroup, memory awareness
- Auto-heap-resizing
- Startup latency improvements

JEP 200 **JEP 201**

#1: Governance

v1

Monolithic Java

- Centralized governance
- Sun championed Java development, JCP, expert group, JSR and TCK
- Closed implementation
- Release in 3, 4 years

shift

Acquisition, community, enterprises and workloads pushed for evolution

- Cloud needed fast release cycles
- Ecosystem needed open governance
- Enterprises needed long term support

v25

Open Java

- OpenJDK & OpenJ9!
- Bi-annual release cycles
- Community based JEP workflow
- Shared stewardship (Oracle, IBM, Google, Azul, Amazon)

JEP 1 **JEP 3**

Java@25: what I have learned

- Languages evolve. Through creative engineering
- And in that process, they trade their purity and identity for efficiency and purpose
- Evolutions are driven by shift in businesses and workloads

Call for action

- 👍 Onboard Java, if not already!
- 👍 Evaluate and contribute to JEPs
- 👍 help features evolve, mature and stabilise
- 👍 Onboard OpenJ9, OpenJDK, Open Liberty
- 👍 Explore the code, contribute patches, or propose optimizations

Resources

📎 Java Enhancement Proposals (JEP)
https://openjdk.org/jeps/0

📎 Supersonic subatomic Cloud native Java (Quarkus)
https://quarkus.io

📎 Eclipse OpenJ9 source
https://github.com/eclipse-openj9/openj9

📎 Open, light-weight, Cloud native Java Runtimes (Open Liberty)
https://openliberty.io

📎 Semeru runtimes downloads
https://developer.ibm.com/languages/java/semeru-runtimes/downloads